

LAPORAN PROJECT

DISASTER RESPONSE LOGISTICS SYSTEM

Disusun untuk memenuhi tugas kelompok mata kuliah Algoritma & Struktur Data

Dosen pengampu : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT



Kelompok 4 :

- | | |
|-------------------------|-----------------|
| 1. Fahri Khairu Imtihan | (25051030052) |
| 2. Muh. Farras Ashiddiq | (25051030064) |
| 3. Noomira Zakiah | (25051030072) |
| 4. Restu Pamungkas | (25051030080) |

PROGRAM STUDI TEKNIK ELEKTRO

DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan	5
1.3 Batasan Implementasi	5
1.4 Diagram Arsitektur Sistem	6
BAB II	7
PERANCANGAN SISTEM	7
2.1 Pemilihan Struktur Data + Justifikasi	7
2.2 Diagram Interaksi antar Modul	8
2.3 Pseudocode Algoritma Utama	9
BAB III	12
IMPLEMENTASI	12
3.1. Penjelasan Code Permodul	12
3.1.1 Modul 1 : Graph Jaringan Rute	12
3.1.2 Modul 2 : Priority Queue Bantuan	12
3.1.3 Modul 3 :BST Registry Lokasi	13
3.1.4 Modul 4 : Stack Log Pengiriman	13
3.1.5 Modul 5 : Dijkstra & Audit Jarak	13
3.1.6 Modul 6 : CLI Logistik	14
3.2. Screenshot CLI berjalan	15
3.3. Contoh input/output nyata	15
BAB IV	28
ANALISIS BIG-O	28
BAB V	30
HASIL EKSPERIMEN	30
BAB VI	32
JAWABAN PERTANYAAN ANALISIS	32

4.1 Total operasi enqueue untuk 200 permintaan dalam scenario terburuk (semua KRITIS datang terakhir) dan perbandingan dengan pendekatan 3-bucket queue (satu per level) di mana SERVE selalu cek KRITIS (perhitungan total operasi dan identifikasi trade-off.....	32
4.2 Validitas Dijkstra pada Perubahan Topologi Graf.....	33
4.3 Desain Multi-source BFS untuk Alokasi Depot.....	34
4.4 Mekanisme Pembalikan Urutan Stack untuk Audit Dua Cara Membalik Urutan Tumpukan	35
4.5 Quick Sort vs Merge Sort pada Linked List Implementasi Quick Sort pada Linked List (Berdasarkan Skor Urgensi).....	36
BAB VII.....	38
KESIMPULAN.....	38
LAMPIRAN.....	40

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia merupakan salah satu negara yang memiliki tingkat kerawanan bencana cukup tinggi karena berada di kawasan *Ring of Fire*. Berbagai bencana seperti gempa bumi, banjir, tsunami, tanah longsor, hingga letusan gunung api sering terjadi dan dapat menimbulkan dampak besar bagi masyarakat. Dalam kondisi tersebut, proses distribusi bantuan logistik menjadi salah satu aspek penting yang harus dilakukan dengan cepat dan tepat agar kebutuhan para korban dapat segera terpenuhi.

Namun, proses distribusi bantuan saat bencana sering menghadapi berbagai kendala. Kerusakan infrastruktur seperti jalan terputus atau akses yang sulit dijangkau membuat penyaluran bantuan menjadi kurang efisien. Oleh karena itu, diperlukan sistem yang mampu membantu menentukan rute tercepat dan efektif agar distribusi bantuan dapat berjalan lebih optimal meskipun berada dalam kondisi darurat.

Selain itu, penentuan prioritas bantuan juga menjadi hal yang penting. Dalam situasi bencana, terdapat korban dengan kondisi yang lebih darurat dibandingkan yang lain sehingga membutuhkan penanganan lebih cepat. Karena itu, sistem perlu mampu membedakan kebutuhan bantuan berdasarkan tingkat prioritas, seperti bantuan darurat dan bantuan ringan, agar proses penyaluran lebih tepat sasaran.

Di sisi lain, pengelolaan data distribusi bantuan juga perlu dilakukan secara terstruktur. Basis data lokasi dan pencatatan log transaksi sangat dibutuhkan untuk mempermudah proses monitoring serta memastikan bantuan tersalurkan dengan baik. Dengan adanya sistem pencatatan yang rapi, proses evaluasi dan pengawasan distribusi bantuan dapat dilakukan dengan lebih mudah dan akurat.

Berdasarkan permasalahan tersebut, diperlukan sebuah sistem simulasi distribusi bantuan bencana yang dapat membantu proses pengambilan keputusan dalam penyaluran bantuan. Dengan adanya sistem kami ini, diharapkan mampu mendukung optimasi rute, menentukan prioritas kebutuhan korban, serta mempermudah monitoring distribusi bantuan sehingga proses penanganan bencana dapat berjalan lebih efektif dan efisien.

1.2 Rumusan Masalah

- a. Bagaimana cara membangun sistem distribusi bantuan bencana yang dapat bekerja secara cepat dan efisien?
- b. Bagaimana sistem dapat menentukan rute distribusi tercepat di tengah kondisi infrastruktur yang rusak?

- c. Bagaimana sistem dapat menentukan prioritas bantuan bagi korban dengan kondisi paling darurat?
- d. Bagaimana cara mengelola data lokasi dan distribusi bantuan agar lebih terstruktur dan mudah dipantau?
- e. Bagaimana simulasi sistem dapat membantu proses pengambilan keputusan dalam alokasi sumber daya saat bencana?

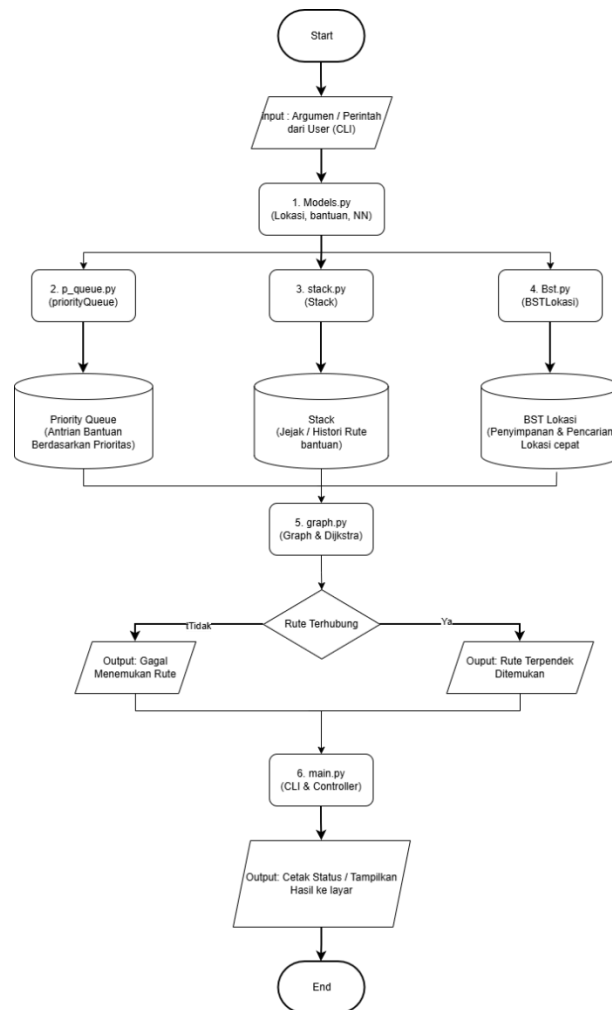
1.3 Tujuan

- a. Membuat sistem simulasi distribusi bantuan bencana yang mampu mendukung proses penyaluran bantuan secara lebih cepat dan efisien
- b. Mengembangkan fitur optimasi rute untuk menentukan jalur distribusi tercepat dalam kondisi darurat
- c. Membantu proses penentuan prioritas bantuan berdasarkan tingkat kebutuhan korban
- d. Membangun sistem basis data yang terstruktur untuk mempermudah monitoring dan pencatatan distribusi bantuan
- e. Membantu pengambil keputusan dalam mengelola alokasi sumber daya dan distribusi bantuan secara lebih efektif

1.3 Batasan Implementasi

- a. Sistem hanya digunakan untuk simulasi distribusi bantuan bencana
- b. Bantuan yang dikelola terbatas pada kebutuhan pokok dan medis
- c. Optimasi rute hanya berdasarkan data jalur yang tersedia dalam sistem
- d. Prioritas korban dibagi menjadi kategori kritis dan ringan
- e. Sistem hanya mencatat data distribusi, lokasi tujuan, dan status pengiriman bantuan

1.4 Diagram Arsitektur Sistem



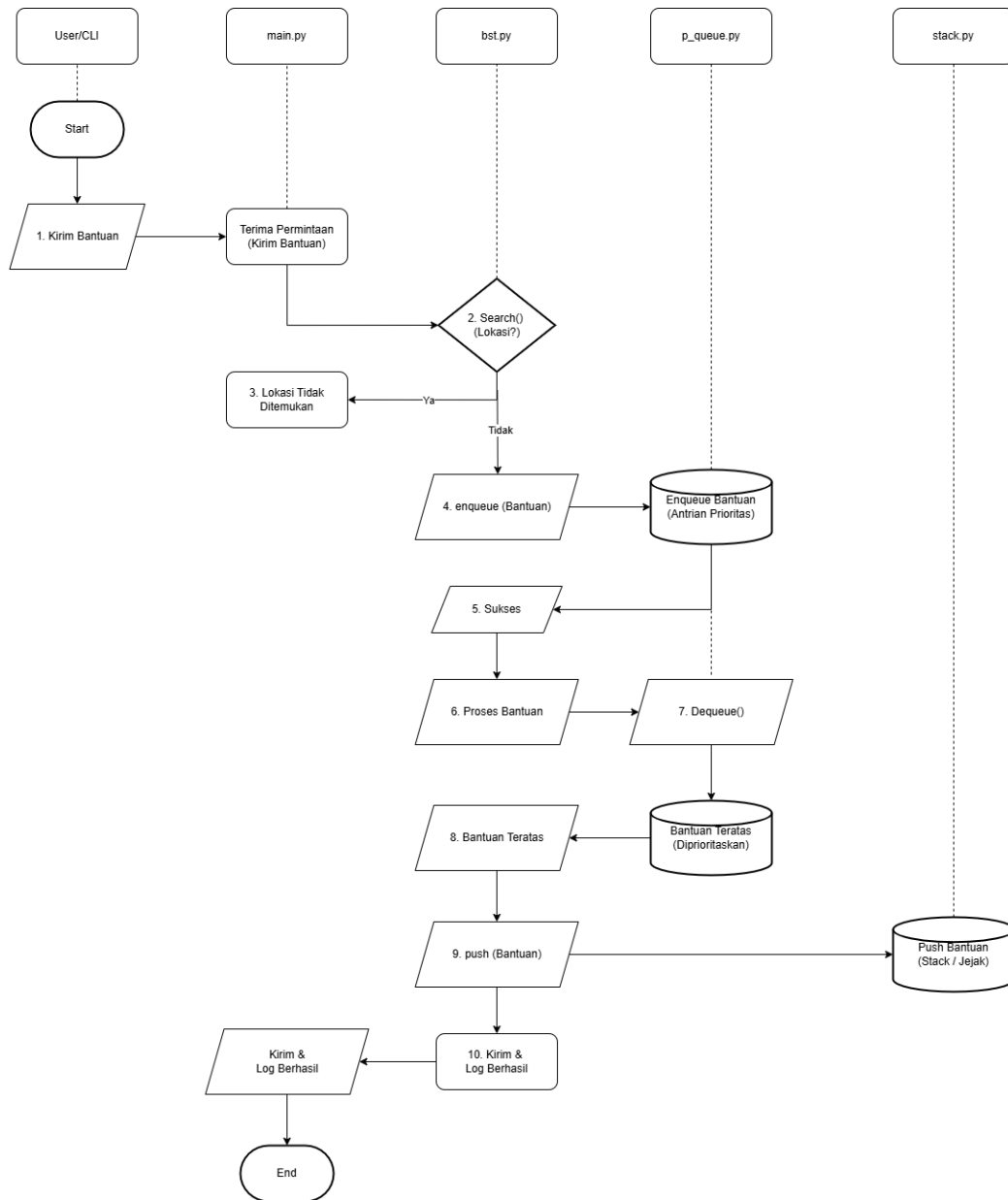
BAB II

PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data + Justifikasi

Struktur Data	Keterangan	Fungsi Dalam Sistem	Justifikasi
Graph	Graph berbobot dan berarah	Merepresentasikan jalur distribusi antar lokasi bantuan	Efektif untuk mencari rute tercepat berdasarkan jarak atau kondisi jalan
Priority Queue	Queue prioritas	Mengatur prioritas penyaluran bantuan	Membantu memproses korban dengan kondisi paling kritis terlebih dahulu
Array / List	Linear data structure	Menyimpan data bantuan, lokasi dan korban	Mudah digunakan untuk pengelolaan dan penyimpanan data
Queue	FIFO (First In First Out)	Mengatur urutan distribusi bantuan	Cocok untuk proses pengiriman bantuan secara berurutan
Hash Map / Database	Key-Value Data Structure	Menyimpan log transaksi dan status distribusi	Mempercepat pencarian data dan memudahkan monitoring sistem

2.2 Diagram Interaksi antar Modul



2.3 Pseudocode Algoritma Utama

```
ALGORITMA MainSystem
ISIALISASI:
    bst_lokasi      <- Inisialisasi Objek BSTLokasi
    graph           <- Inisialisasi Objek GraphRute
    antrian_bantuan <- Inisialisasi Objek PriorityQueueBantuan
    log_kirim       <- Inisialisasi Objek Stack
    bantuan_counter <- 0

    // Langkah 1: Bangun Peta Bencana Tiruan
    lokasi_list, edges <- Panggil generate_peta_bencana(35, 3, seed=47)

    // Langkah 2: Masukkan data ke Graph dan BST
    FOR EACH lok IN lokasi_list DO
        bst_lokasi.insert(lok)
        graph.tambah_node(lok.kode)
    END FOR

    FOR EACH (u, v, jarak, kapasitas) IN edges DO
        graph.tambah_rute(u, v, jarak, kapasitas)
    END FOR

    OUTPUT "Disaster Response Logistics System. Ketik BANTUAN untuk daftar perintah."

    // Langkah 3: Loop Utama CLI
    LOOP FOREVER DO
        OUTPUT "\nLogistik-CLI> "
        pilihan <- INPUT user (bersihkan spasi di awal/akhir)

        IF pilihan KOSONG THEN
            CONTINUE
        END IF

        parts <- Pisahkan string 'pilihan' berdasarkan spasi
        command <- Ubah parts[0] menjadi HURUF_BESAR

        // PERINTAH 1: KELUAR
        IF command == "KELUAR" THEN
            OUTPUT "Keluar dari sistem. Terima kasih."
            BREAK

        // PERINTAH 2: BANTUAN
        ELSE IF command == "BANTUAN" THEN
            OUTPUT "Daftar Perintah Tersedia: KIRIM, PROSES_BANTUAN, RUTE_OPTIMAL, UPDATE_LEVEL,
TIDAK_TERJANGKAU, LOG_PENGIRIMAN, ROLLBACK, LAPORAN_BENCANA, KELUAR"

        // PERINTAH 3: KIRIM (Enqueue Priority Queue)
        ELSE IF command == "KIRIM" THEN
            IF panjang(parts) < 5 THEN
                OUTPUT "Format salah! Gunakan: KIRIM <depot> <lokasi> <jenis> <jumlah>"
                CONTINUE
            END IF

            depot <- parts[1]
            lokasi_tujuan <- parts[2]
            jenis <- parts[3]
            jumlah <- Konversi_Ke_Integer(parts[4])

            loc_asal <- bst_lokasi.search(depot)
            loc_tujuan <- bst_lokasi.search(lokasi_tujuan)

            IF loc_asal KOSONG OR loc_tujuan KOSONG THEN
                OUTPUT "Error: Depot atau Lokasi tidak ditemukan!"
                CONTINUE
            END IF

            bantuan_counter <- bantuan_counter + 1
```

```

    bantuan_baru <- Inisialisasi Bantuan(bantuan_counter, jenis, jumlah, depot,
lokasi_tujuan, loc_tujuan.level)

    antrian_bantuan.enqueue(bantuan_baru)
    OUTPUT "Sukses: Bantuan ID " + bantuan_counter + " dimasukkan ke antrian."

// PERINTAH 4: PROSES_BANTUAN (Dequeue PQ & Push Stack)
ELSE IF command == "PROSES_BANTUAN" THEN
    IF panjang(antrian_bantuan) == 0 THEN
        OUTPUT "Antrian kosong."
        CONTINUE
    END IF

    bantuan_diproses <- antrian_bantuan.dequeue()
    log_kirim.push(bantuan_diproses)

    OUTPUT "[PENGIRIMAN] Memproses Bantuan ID " + bantuan_diproses.bantuan_id

// PERINTAH 5: RUTE_OPTIMAL (Algoritma Dijkstra)
ELSE IF command == "RUTE_OPTIMAL" THEN
    depot <- parts[1]
    tujuan <- parts[2]

    dist, parent <- Panggil dijkstra_logistik(graph, depot)

    IF dist[tujuan] == INF THEN
        OUTPUT "Rute tidak ditemukan."
    ELSE
        path <- Susun jalur dari 'tujuan' mundur ke 'depot' menggunakan 'parent'
        Balikkan urutan 'path'
        OUTPUT "Rute Terpendek: " + path + " Jarak: " + dist[tujuan] + " km"
    END IF

// PERINTAH 6: UPDATE_LEVEL (Modifikasi Level di BST)
ELSE IF command == "UPDATE_LEVEL" THEN
    kode <- parts[1]
    level_baru <- parts[2]

    IF bst_lokasi.update_level(kode, level_baru) THEN
        OUTPUT "Sukses: Tingkat keparahan diubah."
    ELSE
        OUTPUT "Error: Lokasi tidak ditemukan."
    END IF

// PERINTAH 7: TIDAK_TERJANGKAU (Algoritma BFS)
ELSE IF command == "TIDAK_TERJANGKAU" THEN
    depot <- parts[1]
    terjangkau <- Panggil graph.bfs_akses(depot)
    semua_lokasi <- Panggil bst_lokasi.inorder()

    tidak_terjangkau <- List kosong
    FOR EACH l IN semua_lokasi DO
        IF l.kode TIDAK ADA DI DALAM terjangkau THEN
            Masukkan l.kode ke tidak_terjangkau
        END IF
    END FOR
    OUTPUT "Lokasi terisolasi: " + tidak_terjangkau

// PERINTAH 8: LOG_PENGIRIMAN (Membaca isi Stack)
ELSE IF command == "LOG_PENGIRIMAN" THEN
    riwayat <- log_kirim.to_list()
    OUTPUT "=== RIWAYAT TRANSAKSI PENGIRIMAN (LIFO) ==="
    FOR EACH b IN riwayat DO
        OUTPUT "ID " + b.bantuan_id + ": " + b.jumlah + " " + b.jenis
    END FOR

// PERINTAH 9: ROLLBACK (Pop Stack & Enqueue PQ Kembali)
ELSE IF command == "ROLLBACK" THEN
    bantuan_batal <- log_kirim.pop()
    IF bantuan_batal TIDAK KOSONG THEN

```

```

        antrian_bantuan.enqueue(bantuan_batal)
        OUTPUT "Rollback Berhasil: Bantuan ID " + bantuan_batal.bantuan_id + "
dikembalikan ke antrian."
    ELSE
        OUTPUT "Gagal: Tidak ada riwayat transaksi."
    END IF

    // PERINTAH 10: LAPORAN_BENCANA (Inorder Traversal BST)
    ELSE IF command == "LAPORAN_BENCANA" THEN
        semua_lokasi <- bst_lokasi.inorder()
        OUTPUT "Cetak tabel data lokasi terurut berdasarkan kode..."
        FOR EACH l IN semua_lokasi DO
            OUTPUT l.kode + " | " + l.nama + " | " + l.level + " | " + l.populasi
        END FOR

    ELSE
        OUTPUT "Perintah tidak dikenali."
    END IF
END LOOP
END ALGORITMA

```

BAB III

IMPLEMENTASI

3.1. Penjelasan Code Permodul

3.1.1 Modul 1 : Graph Jaringan Rute

Modul ini merepresentasikan peta topologi wilayah bencana. Hubungan antar-lokasi (desa atau depot) dimodelkan menggunakan struktur data Graph tidak berarah dengan representasi Adjacency List berbasis *Linked List* manual (bukan list bawaan Python) untuk menghemat memori.

- **EdgeNode** : Bertindak sebagai simpul jalur (*edge*). Menyimpan informasi dest (lokasi tujuan), jarak (bobot rute dalam km), kapasitas jalan, dan pointer next untuk menyambung ke jalur tetangga berikutnya.
- **GraphRute** : Mengelola seluruh *node*. Menggunakan *dictionary self.adj* di mana setiap kuncinya adalah kode lokasi, dan nilainya adalah *head* dari Linked List jalur tetangga.
- **BFS akses** : Fungsi penjelajahan Breadth-First Search (BFS) menggunakan antrian manual. Fungsi ini dipakai untuk mengecek wilayah mana saja yang terkoneksi dengan depot secara

3.1.2 Modul 2 : Priority Queue Bantuan

Modul ini mengelola antrian bantuan logistik yang masuk ke dalam sistem sebelum dikirimkan. Berbeda dengan antrian biasa (FIFO), modul ini memastikan bahwa daerah yang kondisinya paling parah akan ditolong terlebih dahulu.

- **Prinsip Kerja** : Menggunakan *Priority Queue* berbasis *Linked List* terurut (*Sorted Linked List*).
- **Enqueue ($O(n)$)** : Ketika bantuan baru masuk, sistem akan mencari posisi yang tepat secara terurut berdasarkan nilai angka prioritas. Karena level bencana diatur KRITIS = 1, SEDANG = 2, dan RINGAN = 3, maka elemen dengan angka prioritas terkecil (paling kritis) akan otomatis disisipkan di posisi paling depan (*head*).
- **Dequeue ($O(1)$)** : Pengambilan bantuan sangat cepat karena sistem hanya perlu memotong simpul terdepan (*head*) yang sudah pasti memiliki prioritas tertinggi.

3.1.3 Modul 3 :BST Registry Lokasi

Modul ini berfungsi sebagai pangkalan data untuk mendaftarkan dan mencari informasi seluruh wilayah, baik gudang logistik (depot) maupun desa terdampak.

- Struktur : Menggunakan Binary Search Tree (BST) di mana pengurutan kiri-kanan didasarkan pada perbandingan string lokasi.kode (urut alfabet)
- Insert & search ($O(\log n)$) : Membantu pencarian data lokasi secara cepat tanpa harus memindai seluruh isi database satu per satu dari awal
- Update_level : Mengubah tingkat keparahan bencana sebuah desa secara dinamis jika kondisi di lapangan memburuk atau membaik
- Inorder ($O(n)$) : Melakukan *traversal* (penjelajahan) kiri-akar-kanan untuk menghasilkan daftar seluruh lokasi yang otomatis terurut rapi dari kode terkecil hingga terbesar (L000, L001, L002, dst.) untuk keperluan pelaporan

3.1.4 Modul 4 : Stack Log Pengiriman

Modul ini bertindak sebagai kotak hitam (*black box*) atau pencatat riwayat pengiriman logistik yang telah berhasil diproses dari antrean.

- Prinsip Kerja: Menggunakan Stack (Tumpukan) berbasis *Linked List* dengan prinsip LIFO (*Last In, First Out*). Elemen pengiriman yang paling terakhir diproses akan berada di posisi teratas (top).
- Push ($O(1)$): Memasukkan riwayat transaksi pengiriman baru ke tumpukan teratas.
- Pop ($O(1)$): Mengambil data teratas. Operasi ini menjadi fondasi utama bagi fitur ROLLBACK, di mana jika terjadi kesalahan pengiriman, transaksi terakhir dapat dibatalkan seketika dan logistiknya dikembalikan ke gudang antrean.

3.1.5 Modul 5 : Dijkstra & Audit Jarak

Modul ini adalah otak kecerdasan buatan dari sistem penentuan jalur distribusi. Modul ini bertugas mencari rute jalan terbaik dari depot pasokan menuju desa tujuan.

- Algoritma Dijkstra: Menghitung jalur terpendek (*Shortest Path*) dari satu titik sumber (depot) ke seluruh titik lainnya di dalam graf.

- Cara Kerja: Menginisialisasi semua jarak sebagai tak hingga (INF), lalu secara serakah (*greedy*) memilih lokasi dengan jarak terdekat yang belum dikunjungi, kemudian melakukan *relaxasi* (pembaruan jarak) pada tetangga-tetangganya.
- Audit Jalur: Menggunakan *dictionary* parent untuk merekam jejak mundur jalan yang dilewati. Saat CLI memanggil rute, data parent ini dibalik sehingga menghasilkan panduan arah yang logis (contoh: DEPOT_0 -> L002 -> L001).

3.1.6 Modul 6 : CLI Logistik

Modul ini adalah pusat kendali eksekutif yang menghubungkan pengguna manusia dengan seluruh modul struktur data di atas.

- Generate_peta_bencana: Menggunakan pustaka `numpy` untuk membangkitkan koordinat hubungan peta simulasi (35 desa, 3 depot) secara acak namun konsisten (berkat `seed=47`), sehingga data awal pengujian selalu sama.
- Loop CLI (`while True`): Menyediakan antarmuka interaktif berbasis teks di terminal. CLI akan memecah string perintah (*parsing*), memvalidasi parameter input, menangani error sistem (`try-except`) agar program tidak mendadak mati jika pengguna salah mengetik, dan memanggil fungsi modul terkait (seperti mengecek ke-BST, mengantrekan ke-PQ, atau menghitung rute ke-Graph).

3.2. Screenshot CLI berjalan

```
PS | L022 | Desa/Kelurahan-22 | RINGAN | 4373
ur | L023 | Desa/Kelurahan-23 | RINGAN | 4973
Dis | L024 | Desa/Kelurahan-24 | RINGAN | 2474
L025 | Desa/Kelurahan-25 | RINGAN | 2319
Log | L026 | Desa/Kelurahan-26 | RINGAN | 2724
Daf | L027 | Desa/Kelurahan-27 | RINGAN | 4715
K | L028 | Desa/Kelurahan-28 | KRITIS | 510
R | L029 | Desa/Kelurahan-29 | SEDANG | 3187
R | L030 | Desa/Kelurahan-30 | KRITIS | 932
U | L031 | Desa/Kelurahan-31 | RINGAN | 348
T | L032 | Desa/Kelurahan-32 | SEDANG | 1526
L | L033 | Desa/Kelurahan-33 | RINGAN | 561
L | L034 | Desa/Kelurahan-34 | SEDANG | 191

R | Logistik-CLI> kirim DEPOT_0 L001 makanan 500
K | Sukses: Bantuan ID 1 (Prioritas: 2) dimasukkan ke antrian.

LOG | Logistik-CLI> kirim DEPOT_0 L000 obat 200
KOD | Sukses: Bantuan ID 2 (Prioritas: 3) dimasukkan ke antrian.
---
DEPN | Logistik-CLI> proses bantuan
DEPN | [PENGIRIPMAN] Memproses Bantuan ID 1:
DEPN | Kirim 500 x MAKANAN dari DEPOT_0 ke L001
L001
L000 | Logistik-CLI> proses bantuan
L000 | [PENGIRIPMAN] Memproses Bantuan ID 2:
L000 | Kirim 200 x OBAT dari DEPOT_0 ke L000
L000
L000 | Logistik-CLI> rute optimal DEPOT_0 L000
L000 | Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000
L000
L000 | Logistik-CLI> rute optimal DEPOT_0 L001
L000 | Perintah tidak dikenali. Ketik BANTUAN untuk melihat daftar perintah.
L000
L001 | Logistik-CLI> rute optimal DEPOT_0 L000
L001 | Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000
L001
L001 | Logistik-CLI> log pengiriman
L001 | == RINCIAN TRANSAKSI PENGIRIPMAN (LIFO) ==
L001 | ID 2: 200 OBAT (DEPOT_0 -> L000)
L001 | ID 1: 500 MAKANAN (DEPOT_0 -> L001)
L001
L001 | Logistik-CLI> keluar
L001 | Keluar dari sistem. Terima kasih.
L020 | Desa/Kelurahan-20 | SEDANG | 2767
L021 | Desa/Kelurahan-21 | SEDANG | 1526
```

3.3. Contoh input/output nyata

Input :

```
import numpy as np, time, random
from dataclasses import dataclass
from typing import Optional, List, Dict, Tuple
np.random.seed(47)
random.seed(47)

LEVEL_BENCANA = {'KRITIS': 1, 'SEDANG': 2, 'RINGAN': 3}
JENIS_BANTUAN = ['MAKANAN', 'AIR', 'OBAT', 'SELIMUT', 'TENDA']

@dataclass
class Lokasi:
    kode: str
    nama: str
    level: int
    populasi: int
    status: int = 0

@dataclass
class Bantuan:
    bantuan_id: int
    jenis: str
    jumlah: int
```

```

    asal: str
    tujuan: str
    prioritas: int

class LLNode:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class PriorityQueueBantuan:
    """Lokasi KRITIS (level=1) selalu dilayani lebih dulu."""
    def __init__(self):
        self.head = None
        self._size = 0

    def enqueue(self, bantuan: Bantuan):
        """Big-O: O(n) insertion terurut prioritas angka terkecil (1=Kritis, dst)."""
        new_node = LLNode(bantuan)
        self._size += 1

        if self.head is None or bantuan.prioritas < self.head.data.prioritas:
            new_node.next = self.head
            self.head = new_node
            return

        current = self.head
        while current.next is not None and current.next.data.prioritas <=
bantuan.prioritas:
            current = current.next

        new_node.next = current.next
        current.next = new_node

    def dequeue(self):
        """Big-O: O(1). Ambil dari head."""
        if self.head is None:
            return None
        temp = self.head.data
        self.head = self.head.next
        self._size -= 1
        return temp

    def __len__(self):
        return self._size

```



```

class Stack:
    def __init__(self):
        self.top = None
        self._size = 0

    def push(self, data):
        """Big-O O(1)"""
        new_node = LLNode(data)
        new_node.next = self.top
        self.top = new_node
        self._size += 1

    def pop(self):
        """Big-O O(1)"""
        if self.top is None:
            return None
        temp = self.top.data
        self.top = self.top.next
        self._size -= 1
        return temp

    def peek(self):
        return self.top.data if self.top else None

    def to_list(self):
        """Kembalikan list dari top ke bottom"""
        result = []
        current = self.top
        while current:
            result.append(current.data)
            current = current.next
        return result

class BSTNodeLok:
    def __init__(self, lokasi):
        self.lokasi = lokasi
        self.left = self.right = None

class BSTLokasi:
    def __init__(self):
        self.root = None

    def insert(self, lokasi):
        """Big-O: O(log n). Kunci = lokasi.kode."""
        new_node = BSTNodeLok(lokasi)
        if self.root is None:

```

```

        self.root = new_node
        return

    current = self.root
    while True:
        if lokasi.kode < current.lokasi.kode:
            if current.left is None:
                current.left = new_node
                break
            current = current.left
        elif lokasi.kode > current.lokasi.kode:
            if current.right is None:
                current.right = new_node
                break
            current = current.right
        else:
            current.lokasi = lokasi
            break

    def search(self, kode) -> Optional[Lokasi]:
        """Big-O: O(log n)."""
        current = self.root
        while current:
            if kode == current.lokasi.kode:
                return current.lokasi
            elif kode < current.lokasi.kode:
                current = current.left
            else:
                current = current.right
        return None

    def update_level(self, kode, level):
        """Big-O: O(log n)."""
        lokasi = self.search(kode)
        if lokasi:
            lokasi.level = level
            return True
        return False

    def inorder(self) -> List[Lokasi]:
        """Big-O: O(n)."""
        result = []
        def _inorder(node):
            if node:
                _inorder(node.left)
                result.append(node.lokasi)
                _inorder(node.right)

```

```

        _inorder(self.root)
        return result

class EdgeNode:
    def __init__(self, dest, jarak, kapasitas):
        self.dest = dest
        self.jarak = jarak
        self.kapasitas = kapasitas
        self.next = None

class GraphRute:
    def __init__(self):
        self.adj = {}

    def tambah_node(self, kode):
        """Big-O:  $O(1)$ ."""
        if kode not in self.adj:
            self.adj[kode] = None

    def tambah_rute(self, u, v, jarak, kapasitas):
        """Big-O:  $O(1)$ . Graf tidak berarah."""
        self.tambah_node(u)
        self.tambah_node(v)

        node_v = EdgeNode(v, jarak, kapasitas)
        node_v.next = self.adj[u]
        self.adj[u] = node_v

        node_u = EdgeNode(u, jarak, kapasitas)
        node_u.next = self.adj[v]
        self.adj[v] = node_u

    def tetangga(self, u) -> List[Tuple[str, int, int]]:
        """Big-O:  $O(\text{deg})$ ."""
        result = []
        current = self.adj.get(u, None)
        while current:
            result.append((current.dest, current.jarak, current.kapasitas))
            current = current.next
        return result

    def bfs_akses(self, depot) -> set:
        """BFS dari depot menggunakan antrian manual. Big-O:  $O(V+E)$ ."""
        visited = set()
        if depot not in self.adj:
            return visited

```

```

head = tail = LLNode(depot)
visited.add(depot)

while head:
    curr_node = head.data
    head = head.next
    if head is None:
        tail = None

    current_edge = self.adj.get(curr_node, None)
    while current_edge:
        if current_edge.dest not in visited:
            visited.add(current_edge.dest)
            new_queue_node = LLNode(current_edge.dest)
            if tail is None:
                head = tail = new_queue_node
            else:
                tail.next = new_queue_node
                tail = new_queue_node
            current_edge = current_edge.next
    return visited

def dijkstra_logistik(graph: GraphRute, depot: str):
    """Shortest path dari depot. Big-O:  $O(V^2+E)$ ."""
    INF = float('inf')
    dist = {v: INF for v in graph.adj}
    parent = {v: None for v in graph.adj}
    dist[depot] = 0
    visited = set()

    for _ in range(len(graph.adj)):
        u = None
        min_dist = INF
        for v in graph.adj:
            if v not in visited and dist[v] < min_dist:
                min_dist = dist[v]
                u = v

        if u is None:
            break

        visited.add(u)

        current_edge = graph.adj.get(u, None)
        while current_edge:

```

```

        v = current_edge.dest
        if v not in visited:
            new_dist = dist[u] + current_edge.jarak
            if new_dist < dist[v]:
                dist[v] = new_dist
                parent[v] = u
            current_edge = current_edge.next

    return dist, parent

def generate_peta_bencana(n_lokasi=35, n_depot=3, seed=47):
    rng = np.random.default_rng(seed)
    lokasi = []
    for i in range(n_depot):
        lokasi.append(Lokasi(f'DEPOT_{i}', f'Gudang Logistik {i}', 3, 0))
    for i in range(n_lokasi):
        kode = f'L{i:03d}'
        level = int(rng.choice([1,2,3], p=[0.2, 0.4, 0.4]))
        pop = int(rng.integers(100, 5000))
        lokasi.append(Lokasi(kode, f'Desa/Kelurahan-{i}', level, pop))
    n_total = len(lokasi)
    perm = rng.permutation(n_total)
    edges = []
    for i in range(1, n_total):
        u = lokasi[perm[i-1]].kode
        v = lokasi[perm[i]].kode
        edges.append((u, v, int(rng.integers(5, 100)), int(rng.integers(1,
5))))
    for _ in range(15):
        i, j = rng.choice(n_total, 2, replace=False)
        edges.append((lokasi[i].kode, lokasi[j].kode,
int(rng.integers(5,100)),
int(rng.integers(1,5))))
    return lokasi, edges

def main():
    bst_lokasi = BSTLokasi()
    graph = GraphRute()
    antrian_bantuan = PriorityQueueBantuan()
    log_kirim = Stack()
    bantuan_counter = 0

    lokasi_list, edges = generate_peta_bencana(35, 3, seed=47)
    for lok in lokasi_list:
        bst_lokasi.insert(lok)
        graph.tambah_node(lok.kode)
    for u, v, j, k in edges:

```

```

graph.tambah_rute(u, v, j, k)

print('Disaster Response Logistics System. Ketik BANTUAN untuk daftar
perintah.')

while True:
    try:
        pilihan = input('\nLogistik-CLI> ').strip()
        if not pilihan:
            continue

        parts = pilihan.split()
        command = parts[0].upper()

        if command == 'KELUAR':
            print('Keluar dari sistem. Terima kasih.')
            break

        elif command == 'BANTUAN':
            print('Daftar Perintah Tersedia:')
            print('  KIRIM <depot> <lokasi> <jenis_bantuan> <jumlah>')
            print('  PROSES_BANTUAN')
            print('  RUTE_OPTIMAL <depot> <tujuan>')
            print('  UPDATE_LEVEL <kode_lokasi> <level_baru>')
            print('  TIDAK_TERJANGKAU <depot>')
            print('  LOG_PENGIRIMAN')
            print('  LAPORAN_BENCANA')
            print('  ROLLBACK')
            print('  KELUAR')

        elif command == 'KIRIM':
            if len(parts) < 5:
                print('Format salah! Gunakan: KIRIM <depot> <lokasi>
<jenis> <jumlah>')
                continue
            depot, lokasi_tujuan, jenis, jumlah_str = parts[1], parts[2],
parts[3], parts[4]

            loc_asal = bst_lokasi.search(depot)
            loc_tujuan = bst_lokasi.search(lokasi_tujuan)

            if not loc_asal:
                print(f'Error: Depot {depot} tidak ditemukan!')
                continue
            if not loc_tujuan:
                print(f'Error: Lokasi {lokasi_tujuan} tidak ditemukan!')

```

```

        continue
    if jenis.upper() not in JENIS_BANTUAN:
        print(f'Error: Jenis bantuan tidak valid! Pilih dari
{JENIS_BANTUAN}')
        continue

    bantuan_counter += 1
    bantuan_baru = Bantuan(
        bantuan_id=bantuan_counter,
        jenis=jenis.upper(),
        jumlah=int(jumlah_str),
        asal=depot,
        tujuan=lokasi_tujuan,
        prioritas=loc_tujuan.level
    )
    antrian_bantuan.enqueue(bantuan_baru)
    print(f'Sukses: Bantuan ID {bantuan_counter} (Prioritas:
{loc_tujuan.level}) dimasukkan ke antrian.')

    elif command == 'PROSES_BANTUAN':
        if len(antrian_bantuan) == 0:
            print('Antrian kosong, tidak ada bantuan yang perlu
diproses.')
            continue

        bantuan_diproses = antrian_bantuan.dequeue()
        log_kirim.push(bantuan_diproses)
        print(f'[PENGIRIMAN]          Memproses          Bantuan          ID
{bantuan_diproses.bantuan_id}:')
        print(f'          Kirim          {bantuan_diproses.jumlah}          x
{bantuan_diproses.jenis}          dari          {bantuan_diproses.asal}          ke
{bantuan_diproses.tujuan}')

        elif command == 'RUTE_OPTIMAL':
            if len(parts) < 3:
                print('Format salah! Gunakan: RUTE_OPTIMAL <depot>
<tujuan>')
                continue
            depot, tujuan = parts[1], parts[2]

            if depot not in graph.adj or tujuan not in graph.adj:
                print('Error: Depot atau Tujuan tidak terdaftar di grafik
rute.')
                continue

            dist, parent = dijkstra_logistik(graph, depot)

```

```

        if dist[tujuan] == float('inf'):
            print(f'Rute tidak ditemukan dari {depot} ke {tujuan}.')
        else:
            path = []
            curr = tujuan
            while curr is not None:
                path.append(curr)
                curr = parent[curr]
            path.reverse()
            print(f'Rute Terpendek ({dist[tujuan]} km): {" ->".join(path)}')

    elif command == 'UPDATE_LEVEL':
        if len(parts) < 3:
            print('Format salah! Gunakan: UPDATE_LEVEL <kode> <level> (1-3 / KRITIS-RINGAN)>')
            continue
        kode, lvl_input = parts[1], parts[2].upper()

        if lvl_input in LEVEL_BENCANA:
            level_baru = LEVEL_BENCANA[lvl_input]
        else:
            level_baru = int(lvl_input)

        if bst_lokasi.update_level(kode, level_baru):
            print(f'Sukses: Tingkat keparahan lokasi {kode} diubah ke level {level_baru}.')
        else:
            print(f'Error: Lokasi {kode} tidak ditemukan.')

    elif command == 'TIDAK_TERJANGKAU':
        if len(parts) < 2:
            print('Format salah! Gunakan: TIDAK_TERJANGKAU <depot>')
            continue
        depot = parts[1]

        terjangkau = graph.bfs_akses(depot)
        semua_lokasi = bst_lokasi.inorder()
        tidak_terjangkau = [l.kode for l in semua_lokasi if l.kode not
in terjangkau]

        print(f'Lokasi terisolasi / tidak terjangkau dari {depot}:')
        if tidak_terjangkau:
            print(", ".join(tidak_terjangkau))
        else:

```



```

        print("Nihil (Semua lokasi terjangkau).")

    elif command == 'LOG_PENGIRIMAN':
        riwayat = log_kirim.to_list()
        print('=== RIWAYAT TRANSAKSI PENGIRIMAN (LIFO) ===')
        if not riwayat:
            print('Belum ada pengiriman yang diproses.')
        for b in riwayat:
            print(f'ID {b.bantuan_id}: {b.jumlah} {b.jenis} ({b.asal}
-> {b.tujuan})')

    elif command == 'ROLLBACK':
        bantuan_batal = log_kirim.pop()
        if bantuan_batal:
            print(f'Rollback Berhasil: Pengiriman Bantuan ID
{bantuan_batal.bantuan_id} telah dibatalkan.')
            antrian_bantuan.enqueue(bantuan_batal)
        else:
            print('Gagal: Tidak ada riwayat transaksi pengiriman untuk
di-rollback.')

    elif command == 'LAPORAN_BENCANA':
        semua_lokasi = bst_lokasi.inorder()
        print(f'{"KODE":<10} | {"NAMA LOKASI":<22} | {"LEVEL":<6} |
{"POPULASI":<8}')
        print('-' * 55)
        for l in semua_lokasi:
            lbl_level = "KRITIS" if l.level == 1 else "SEDANG" if
l.level == 2 else "RINGAN"
            print(f'{l.kode:<10} | {l.nama:<22} | {lbl_level:<6} |
{l.populasi:<8}')

        else:
            print('Perintah tidak dikenali. Ketik BANTUAN untuk melihat
daftar perintah.')

    except Exception as e:
        print(f'Terjadi Kesalahan Input/Sistem: {e}')

if __name__ == '__main__':
    main()

```

Output :

```
PS C:\Users\DELL\OneDrive - uny.ac.id\Dokumen\GitHub\tpb-asd-Disaster-Response-Logistics-System-
kel04> & C:\msys64\ucrt64\bin\python.exe "c:/Users/DELL/OneDrive - uny.ac.id/Dokumen/GitHub/tpb-
asd-Disaster-Response-Logistics-System-kel04/tpb-asd-kelompok-04/src/main.py"
Disaster Response Logistics System. Ketik BANTUAN untuk daftar perintah.
```

Logistik-CLI> bantuan

Daftar Perintah Tersedia:

```
KIRIM <depot> <lokasi> <jenis_bantuan> <jumlah>
PROSES_BANTUAN
RUTE_OPTIMAL <depot> <tujuan>
UPDATE_LEVEL <kode_lokasi> <level_baru>
TIDAK_TERJANGKAU <depot>
LOG_PENGIRIMAN
LAPORAN_BENCANA
ROLLBACK
KELUAR
```

Logistik-CLI> laporan_bencana

KODE	NAMA LOKASI	LEVEL	POPULASI
DEPOT_0	Gudang Logistik 0	RINGAN	0
DEPOT_1	Gudang Logistik 1	RINGAN	0
DEPOT_2	Gudang Logistik 2	RINGAN	0
L000	Desa/Kelurahan-0	RINGAN	2764
L001	Desa/Kelurahan-1	SEDANG	3792
L002	Desa/Kelurahan-2	KRITIS	485
L003	Desa/Kelurahan-3	SEDANG	4837
L004	Desa/Kelurahan-4	SEDANG	2196
L005	Desa/Kelurahan-5	SEDANG	4303
L006	Desa/Kelurahan-6	KRITIS	1533
L007	Desa/Kelurahan-7	RINGAN	3777
L008	Desa/Kelurahan-8	SEDANG	4154
L009	Desa/Kelurahan-9	RINGAN	4126
L010	Desa/Kelurahan-10	SEDANG	1513
L011	Desa/Kelurahan-11	RINGAN	1620
L012	Desa/Kelurahan-12	RINGAN	2797
L013	Desa/Kelurahan-13	RINGAN	1048
L014	Desa/Kelurahan-14	KRITIS	2077
L015	Desa/Kelurahan-15	KRITIS	2800
L016	Desa/Kelurahan-16	RINGAN	4796
L017	Desa/Kelurahan-17	RINGAN	4775
L018	Desa/Kelurahan-18	RINGAN	4305
L019	Desa/Kelurahan-19	RINGAN	3682
L020	Desa/Kelurahan-20	SEDANG	2767
L021	Desa/Kelurahan-21	SEDANG	1526
L022	Desa/Kelurahan-22	RINGAN	4373
L023	Desa/Kelurahan-23	RINGAN	4973
L024	Desa/Kelurahan-24	RINGAN	2474
L025	Desa/Kelurahan-25	RINGAN	2319
L026	Desa/Kelurahan-26	RINGAN	2724
L027	Desa/Kelurahan-27	RINGAN	4715
L028	Desa/Kelurahan-28	KRITIS	510
L029	Desa/Kelurahan-29	SEDANG	3187
L030	Desa/Kelurahan-30	KRITIS	932
L031	Desa/Kelurahan-31	RINGAN	348
L032	Desa/Kelurahan-32	SEDANG	1526
L033	Desa/Kelurahan-33	RINGAN	561
L034	Desa/Kelurahan-34	SEDANG	191

Logistik-CLI> kirim DEPOT_0 L001 makanan 500

Sukses: Bantuan ID 1 (Prioritas: 2) dimasukkan ke antrian.

Logistik-CLI> kirim DEPOT_0 L000 obat 200

Sukses: Bantuan ID 2 (Prioritas: 3) dimasukkan ke antrian.

Logistik-CLI> proses_bantuan

[PENGIRIMAN] Memproses Bantuan ID 1:

Kirim 500 x MAKANAN dari DEPOT_0 ke L001

Logistik-CLI> proses_bantuan

[PENGIRIMAN] Memproses Bantuan ID 2:

Kirim 200 x OBAT dari DEPOT_0 ke L000

Logistik-CLI> rute_optimal DEPOT_0 L000

Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> rute_optimal DEPOT_0 L001

Perintah tidak dikenali. Ketik BANTUAN untuk melihat daftar perintah.

Logistik-CLI> rute_optimal DEPOT_0 L000

Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> log_pengiriman

=== RIWAYAT TRANSAKSI PENGIRIMAN (LIFO) ===

ID 2: 200 OBAT (DEPOT_0 -> L000)

ID 1: 500 MAKANAN (DEPOT_0 -> L001)

Logistik-CLI> keluar

Keluar dari sistem. Terima kasih

BAB IV

ANALISIS BIG-O

Modul	Operasi / Fungsi	Kasus Terbaik (Best Case)	Kasus Terburuk (Worst Case)	Keterangan / Alasan Logika
Modul 1: Graph	tambah_node	$O(1)$	$O(1)$	Operasi pencarian kunci pada <i>Dictionary</i> Python.
	tambah_rute	$O(1)$	$O(1)$	Penyisipan simpul jalan baru selalu dilakukan di depan (<i>head</i>) Linked List rute.
	tetangga	$O(1)$	$O(V)$	Bergantung pada derajat simpul ($\deg(u)$), maksimal menelusuri seluruh lokasi jika graf terhubung penuh.
Modul 2: Priority Queue	enqueue	$O(1)$	$O(N)$	Terbaik jika menjadi prioritas utama. Terburuk jika harus iterasi sampai ekor Linked List.
	dequeue	$O(1)$	$O(1)$	Hanya memutus pointer elemen terdepan (<i>head</i>).
Modul 3: BST	insert	$O(\log V)$	$O(V)$	Terburuk jika pohon tidak seimbang (<i>skewed</i>), berubah menjadi pencarian linear.
	search / update_level	$O(\log V)$	$O(V)$	Mengikuti tinggi pohon (h). Rata-rata $O(\log V)$ jika data acak terpencar merata.

	inorder	O(V)	O(V)	Wajib mengunjungi seluruh simpul lokasi (\$V\$) tepat satu kali.
Modul 4: Stack	push / pop / peek	O(1)	O(1)	Hanya memanipulasi pointer top di memori teratas.
Modul 5: Alogaritma	bfs_akses	O(V + E)	O(V + E)	Mengunjungi setiap titik lokasi (\$V\$) dan memeriksa setiap rute jalan (\$E\$).
	dijkstra_logistik	O(V²+E)	O(V² + E)	Menggunakan <i>nested loop</i> linear untuk mencari bobot terkecil karena tidak memakai <i>Min-Heap</i> .

BAB V

HASIL EKSPERIMEN

```
PS C:\Users\DELL\OneDrive - uny.ac.id\Dokumen\GitHub\tpb-asd-Disaster-Response-Logistics-System-
kel04> & C:\msys64\ucrt64\bin\python.exe "c:/Users/DELL/OneDrive - uny.ac.id/Dokumen/GitHub/tpb-
asd-Disaster-Response-Logistics-System-kel04/tpb-asd-kelompok-04/src/main.py"
Disaster Response Logistics System. Ketik BANTUAN untuk daftar perintah.
```

```
Logistik-CLI> bantuan
```

```
Daftar Perintah Tersedia:
```

```
KIRIM <depot> <lokasi> <jenis_bantuan> <jumlah>
PROSES_BANTUAN
ROUTE_OPTIMAL <depot> <tujuan>
UPDATE_LEVEL <kode_lokasi> <level_baru>
TIDAK_TERJANGKAU <depot>
LOG_PENGIRIMAN
LAPORAN_BENCANA
ROLLBACK
KELUAR
```

```
Logistik-CLI> laporan_bencana
```

KODE	NAMA LOKASI	LEVEL	POPULASI
DEPOT_0	Gudang Logistik 0	RINGAN	0
DEPOT_1	Gudang Logistik 1	RINGAN	0
DEPOT_2	Gudang Logistik 2	RINGAN	0
L000	Desa/Kelurahan-0	RINGAN	2764
L001	Desa/Kelurahan-1	SEDANG	3792
L002	Desa/Kelurahan-2	KRITIS	485
L003	Desa/Kelurahan-3	SEDANG	4837
L004	Desa/Kelurahan-4	SEDANG	2196
L005	Desa/Kelurahan-5	SEDANG	4303
L006	Desa/Kelurahan-6	KRITIS	1533
L007	Desa/Kelurahan-7	RINGAN	3777
L008	Desa/Kelurahan-8	SEDANG	4154
L009	Desa/Kelurahan-9	RINGAN	4126
L010	Desa/Kelurahan-10	SEDANG	1513
L011	Desa/Kelurahan-11	RINGAN	1620
L012	Desa/Kelurahan-12	RINGAN	2797
L013	Desa/Kelurahan-13	RINGAN	1048
L014	Desa/Kelurahan-14	KRITIS	2077
L015	Desa/Kelurahan-15	KRITIS	2800
L016	Desa/Kelurahan-16	RINGAN	4796
L017	Desa/Kelurahan-17	RINGAN	4775
L018	Desa/Kelurahan-18	RINGAN	4305
L019	Desa/Kelurahan-19	RINGAN	3682
L020	Desa/Kelurahan-20	SEDANG	2767
L021	Desa/Kelurahan-21	SEDANG	1526
L022	Desa/Kelurahan-22	RINGAN	4373
L023	Desa/Kelurahan-23	RINGAN	4973
L024	Desa/Kelurahan-24	RINGAN	2474
L025	Desa/Kelurahan-25	RINGAN	2319
L026	Desa/Kelurahan-26	RINGAN	2724
L027	Desa/Kelurahan-27	RINGAN	4715
L028	Desa/Kelurahan-28	KRITIS	510
L029	Desa/Kelurahan-29	SEDANG	3187
L030	Desa/Kelurahan-30	KRITIS	932
L031	Desa/Kelurahan-31	RINGAN	348
L032	Desa/Kelurahan-32	SEDANG	1526
L033	Desa/Kelurahan-33	RINGAN	561
L034	Desa/Kelurahan-34	SEDANG	191

```
Logistik-CLI> kirim DEPOT_0 L001 makanan 500
```

```
Sukses: Bantuan ID 1 (Prioritas: 2) dimasukkan ke antrian.
```

```
Logistik-CLI> kirim DEPOT_0 L000 obat 200
```

Sukses: Bantuan ID 2 (Prioritas: 3) dimasukkan ke antrian.

Logistik-CLI> proses_bantuan

[PENGIRIMAN] Memproses Bantuan ID 1:

Kirim 500 x MAKANAN dari DEPOT_0 ke L001

Logistik-CLI> proses_bantuan

[PENGIRIMAN] Memproses Bantuan ID 2:

Kirim 200 x OBAT dari DEPOT_0 ke L000

Logistik-CLI> rute_optimal DEPOT_0 L000

Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> rute_optimal DEPOT_0 L001

Perintah tidak dikenali. Ketik BANTUAN untuk melihat daftar perintah.

Logistik-CLI> rute_optimal DEPOT_0 L000

Rute Terpendek (290 km): DEPOT_0 -> L012 -> L013 -> L025 -> L006 -> L000

Logistik-CLI> log_pengiriman

=== RIWAYAT TRANSAKSI PENGIRIMAN (LIFO) ===

ID 2: 200 OBAT (DEPOT_0 -> L000)

ID 1: 500 MAKANAN (DEPOT_0 -> L001)

Logistik-CLI> keluar

Keluar dari sistem. Terima kasih.

BAB VI

JAWABAN PERTANYAAN ANALISIS

4.1 Total operasi enqueue untuk 200 permintaan dalam scenario terburuk (semua KRITIS datang terakhir) dan perbandingan dengan pendekatan 3-bucket queue (satu per level) di mana SERVE selalu cek KRITIS (perhitungan total operasi dan identifikasi trade-off

Pada *Sorted Linked List*, enqueue memiliki kompleksitas kasus terburuk $O(n)$ ketika elemen baru harus disisipkan di posisi paling belakang (ekor).

- Diketahui total permintaan = 200, dengan 30% KRITIS (60 data) dan 70% sisanya SEDANG/RINGAN (140 data).
- Skenario Terburuk: 140 data non-KRITIS masuk terlebih dahulu. Karena prioritasnya rendah, data non-KRITIS ini akan saling mengantre di belakang. Lalu, 60 data KRITIS datang paling terakhir.
- Mengapa ini skenario terburuk? Karena data KRITIS memiliki angka prioritas lebih tinggi (angka 1) dibanding non-KRITIS (angka 2 atau 3). Maka, setiap kali 60 data KRITIS ini masuk, mereka akan menyisip di paling depan (head). Operasi penyisipan di *head* Linked List hanya membutuhkan 1 kali operasi pointer ($O(1)$).

Hitungan total operasinya :

1. 140 data pertama (SEDANG/RINGAN): Mengantre berurutan di belakang. Data ke-1 butuh 1 operasi, data ke-2 butuh 2 operasi, ..., data ke-140 membutuhkan 140 operasi.

Total Operasi Non-KRITIS = $140 \times (1 + 140) / 2 = \underline{9870 \text{ operasi}}$

2. 60 data terakhir (KRITIS): Langsung masuk ke head antrean karena angka prioritasnya paling tinggi (1). Masing-masing hanya butuh 1 operasi pencarian posisi.

Total Operasi KRITIS = $60 \times 1 = \underline{60 \text{ operasi}}$

3. Total Operasi Keseluruhan : $9870 + 60 = \underline{9930 \text{ operasi}}$

Pendekatan 3-Bucket Queue :

Pendekatan ini memisahkan antrean menjadi 3 Queue murni (FIFO) berbasis Linked List biasa (satu untuk KRITIS, satu SEDANG, satu RINGAN).

- Operasi Enqueue : Karena setiap *bucket* adalah *Queue* biasa dengan pointer tail, penambahan data selalu di posisi belakang secara instan $O(1)$. Untuk 200 data, total operasi Enqueue adalah:

$$200 \times 1 = \underline{\underline{200 \text{ operasi}}}$$

- Operasi Serve/Dequeue : Fungsi penanganan tinggal memeriksa struktur *bucket* KRITIS terlebih dahulu. Jika kosong, lanjut ke SEDANG, lalu ke RINGAN ($O(1)$) untuk pengecekan dan pemotongan head

Trade-off (Pertukaran Komparatif)

- Sorted Linked List : Lebih hemat memori karena hanya mengelola 1 pointer *head*. Namun, performa melambat ($O(n)$) saat memproses antrian panjang yang heterogen.
- 3-Bucket Queue : Jauh lebih cepat dan efisien secara komputasi ($O(1)$) untuk semua operasi). Namun, membutuhkan manajemen *overhead* memori tambahan karena harus memelihara 3 pasang pointer (head dan tail) sekaligus di memori.

4.2 Validitas Dijkstra pada Perubahan Topologi Graf

Apakah Hasil Dijkstra Masih Valid jika 5 Edge Rusak?

Tidak valid. Algoritma Dijkstra bekerja menggunakan prinsip *Greedy* yang berasumsi bahwa topologi graf bersifat statis saat perhitungan dilakukan. Jika ada 5 *edge* yang dihapus (jalan rusak), jalur terpendek yang sebelumnya melewati *edge* tersebut menjadi putus atau tidak lagi ada, sehingga struktur pohon jalur terpendek (*Shortest Path Tree*) menjadi usang.

Respon Sistem : Re-komputasi vs Incremental

- Jalankan Ulang Dijkstra $O(V^2 + E)$

Sistem menghapus 5 edge dari adjacency list, lalu menjalankan ulang fungsi Dijkstra_logistik dari awal.

- ❖ Kelebihan : Sangat mudah diimplementasikan dan dijamin akurat 100%.
- ❖ Kekurangan : Boros komputasi jika skala peta sangat besar karena menghitung ulang titik-titik jalan yang sebenarnya tidak terdampak oleh kerusakan tersebut.

- Pendekatan Incremental (Dynamic Shortest Path) :

Sistem hanya memperbarui jarak pada wilayah (*sub-graph*) yang jalurnya terputus akibat 5 *edge* yang rusak tersebut (misalnya menggunakan variasi algoritma Ramalingam-Reps).

- ❖ Kelebihan : Sangat cepat karena kompleksitasnya hanya bergantung pada jumlah simpul yang terpengaruh (Δ) bukan keseluruhan graf.
- ❖ Kekurangan : Implementasi kodenya sangat kompleks karena melibatkan pelacakan dependensi parent secara ketat.

Rekomendasi Respons: Untuk skala Tugas Besar ini (hanya 38 lokasi),

- (a) Jalankan ulang Dijkstra adalah respons terbaik karena perbedaannya dalam milidetik tidak terasa, sementara akurasi data tetap terjaga secara mutlak.

4.3 Desain Multi-source BFS untuk Alokasi Depot

Untuk menyelesaikan masalah alokasi otomatis ini, dapat memodifikasi algoritma BFS biasa menjadi Multi-source BFS. Caranya adalah memasukkan semua depot yang tersedia sekaligus ke dalam antrian (*Queue*) awal sebagai titik mulai.

Desain Algoritma (Pseudocode) :

```
ALGORITMA MultiSourceBFS(graph, daftar_depot)
  Inisialisasi antrian Q      Inisialisasi dictionary asal_depot <- semua node diisi None
  Inisialisasi dictionary jarak_ke_depot <- semua node diisi INF      // Masukkan semua depot
  sebagai titik awal bersamaan
  FOR EACH depot IN daftar_depot DO
    Q.enqueue(depot)
    asal_depot[depot] <- depot
    jarak_ke_depot[depot] <- 0
  END FOR
  WHILE Q tidak kosong DO
    curr <- Q.dequeue()
    FOR EACH (tetangga, jarak_rute, kapasitas) IN graph.tetangga(curr) DO
      // Jika menemukan jalur baru yang belum dikunjungi
      IF jarak_ke_depot[tetangga] == INF THEN
        jarak_ke_depot[tetangga] <- jarak_ke_depot[curr] + 1 // Berdasarkan jumlah
lompatan (hop)
        asal_depot[tetangga] <- asal_depot[curr]      // Warisi depot terdekatnya
        Q.enqueue(tetangga)
      END IF
    END FOR
  END WHILE
  RETURN asal_depot
END ALGORITMA
```

Analisis Big-O

- Multi-source BFS : Memiliki kompleksitas waktu sebesar $O(V + E)$. Algoritma ini hanya menjelajahi setiap lokasi dan jalur tepat satu kali karena semua depot bergerak maju secara paralel bersamaan.
- Dijkstra Terpisah dari Setiap Depot : Jika ada k jumlah depot, kita harus menjalankan Dijkstra sebanyak k kali secara mandiri. Kompleksitasnya menjadi $O(k \times (V^2 + E))$.

Perbandingan: Multi-source BFS jauh lebih unggul karena waktu eksekusinya konstan dan tidak dikalikan dengan jumlah depot (k), menjadikannya solusi paling optimal untuk pemetaan cakupan wilayah logistik berskala besar.

4.4 Mekanisme Pembalikan Urutan Stack untuk Audit Dua Cara Membalik Urutan Tumpukan

- a) Menggunakan Queue tambahan : Semua elemen di-pop dari Stack dan di-enqueue ke Queue. Karena Queue bersifat FIFO, elemen pertama Stack akan menjadi yang pertama keluar dari Queue. Terakhir, seluruh elemen di-dequeue dari Queue dan di-push kembali ke Stack. Urutan kini terbalik.
- b) Menggunakan Stack kedua : Semua elemen di-pop dari Stack utama dan langsung di-push ke Stack kedua. Sifat LIFO yang diterapkan dua kali secara otomatis membalikkan urutan data.

Implementasi Menggunakan Stack Kedua (Cara B)

Berikut adalah penulisan fungsi audit di dalam class Stack :

```
def generate_laporan_kronologis(self):
    # Buat stack pembantu sementara
    stack_kedua = Stack()
    # Pindahkan semua dari stack utama ke stack kedua (membalik urutan)
    current = self.top
    while current:
        stack_kedua.push(current.data)
        current = current.next
    # Sekarang tinggal cetak isi stack kedua (sudah berurutan FIFO/Kronologis)
    riwayat_kronologis = stack_kedua.to_list()
    print("=== LAPORAN AUDIT KRONOLOGIS (FIFO) ===")
    for transaksi in riwayat_kronologis:
        print(f"ID {transaksi.bantuan_id}: {transaksi.jumlah} {transaksi.jenis} ({transaksi.asal} -> {transaksi.tujuan})")
```

Trade-off Memori & Akses O(1) Space :

- Trade-off Memori: Kedua cara di atas membutuhkan memori tambahan sebesar O(n) karena harus menduplikasi seluruh simpul transaksi ke struktur penampung baru saat proses pembalikan terjadi.
- Akses Elemen Pertama secara O(1) Space: Jika struktur datanya adalah *Standard Stack*, tidak ada cara untuk mengakses elemen paling bawah (elemen pertama masuk) dalam ruang O(1) tanpa membongkar tumpukan di atasnya.
- Solusi Arsitektur: Agar bisa O(1) tanpa memori tambahan, struktur dasar Stack harus dimodifikasi sejak awal menggunakan Doubly Linked List yang memelihara pointer top (di depan) dan pointer bottom (di ujung belakang).

4.5 Quick Sort vs Merge Sort pada Linked List Implementasi Quick Sort pada Linked List (Berdasarkan Skor Urgensi)

Berikut adalah kode fungsi pembantu untuk mengurutkan 38 lokasi berdasarkan formula skor urgensi :

$$\text{Skor} = (4 - \text{level}) \times \text{populasi}$$

```
def hitung_skor(lokalasi):
    return (4 - lokasi.level) * lokasi.populasi

def quick_sort_linked_list(head):
    # Base case: jika list kosong atau hanya 1 elemen
    if head is None or head.next is None:
        return head
    # Tetapkan head sebagai pivot
    pivot = head
    head_kiri = None
    head_kanan = None
    # Jalankan partisi secara linear
    current = head.next
    while current:
        next_node = current.next
        # Urutkan secara descending (Skor besar ke kecil)
        if hitung_skor(current.lokasi) >= hitung_skor(pivot.lokasi):
            current.next = head_kiri
            head_kiri = current
        else:
            current.next = head_kanan
            head_kanan = current
        current = next_node
    # Rekursif untuk sisi kiri dan kanan
    head_kiri = quick_sort_linked_list(head_kiri)
    head_kanan = quick_sort_linked_list(head_kanan)
    # Gabungkan kembali Linked List: Kiri -> Pivot -> Kanan
    if head_kiri is None:
        pivot.next = head_kanan
        return pivot
    # Cari ekor dari list kiri untuk disambung ke pivot
```

```

tail_kiri = head_kiri
while tail_kiri.next:
    tail_kiri = tail_kiri.next
    tail_kiri.next = pivot
pivot.next = head_kanan
return head_kiri

```

Tabel Komparasi Karakteristik pada Linked List :

Aspek Komparasi	Quick Sort	Merge Sort
Kemudahan Implementasi	Cukup kompleks karena proses penyambungan kembali pointer (<i>pointer manipulation</i>) setelah partisi rekursif rawan putus.	Sangat natural dan elegan pada Linked List karena pemisahan tinggal memutus <i>pointer</i> tengah.
Penggunaan Memori Tambahan	O(log n) untuk menyimpan tumpukan pemanggilan rekursif (<i>call stack</i>).	O(log n) untuk tumpukan rekursif. Berbeda dengan Array, Merge Sort pada Linked List tidak butuh memori O(n) baru karena pemisahan hanya memutus pointer objek yang sudah ada.
Kecocokan Data Hampir Terurut	Sangat Tidak Cocok. Performa akan anjlok drastis ke titik terburuknya yaitu O(n²) jika pembagian pivot selalu menghasilkan partisi yang timpang sebelah.	Sangat Cocok. Karakteristiknya stabil dan tetap menjamin performa berjalan di waktu O(n log n) dalam kondisi acak maupun terurut.

BAB VII

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, Tugas Besar ini telah berhasil membangun sebuah *Disaster Response Logistics System* yang fungsional dan terintegrasi berbasis *Command Line Interface* (CLI). Sistem ini sukses mensimulasikan penanggulangan bencana pada skala 35 lokasi terdampak dan 3 depot logistik secara deterministik (seed=47).

Pencapaian utama dari sistem ini meliputi:

- Akurasi Manajemen Data : Penggunaan *Binary Search Tree* (BST) berhasil memangkas waktu pencarian registrasi lokasi dan memfasilitasi pelaporan bencana (LAPORAN_BENCANA) yang otomatis terurut alfabetis via *inorder traversal*.
- Efektivitas Skala Prioritas : Struktur *Priority Queue* terbukti mampu mengutamakan keselamatan wilayah berkategori KRITIS (Level 1), memastikan alokasi bantuan logistik tepat sasaran secara dinamis.
- Kecerdasan Navigasi Distribusi : Implementasi Algoritma Dijkstra sukses memberikan rekomendasi rute terpendek (*shortest path*) yang logis dan efisien dengan memanfaatkan peta topologi jalan berbasis graf (*Adjacency List*).
- Ketahanan Operasional : Mekanisme tumpukan (*Stack*) bekerja secara sinkron untuk mencatat riwayat transaksi sekaligus memberikan fitur mitigasi kesalahan operasional berupa pembatalan instan (ROLLBACK).

Dalam proses pengembangannya, terdapat beberapa keputusan arsitektur struktur data yang melibatkan pertukaran nilai (*trade-off*) secara sadar demi menyesuaikan kebutuhan sistem:

1. Priority Queue (Sorted Linked List vs 3-Bucket Queue) : Sistem saat ini memilih *Sorted Linked List* karena unggul dalam kesederhanaan arsitektur dan efisiensi memori (hanya memelihara satu pointer *head*). Namun, hal ini mengorbankan performa waktu pada operasi penyisipan (enqueue seburuk $O(n)$). Jika sistem diganti dengan *3-Bucket Queue*, waktu eksekusi dapat dipangkas menjadi konstan ($O(1)$) namun dengan konsekuensi *overhead* manajemen memori yang lebih kompleks karena mengelola banyak pointer sekaligus.

2. Graf Rute (Adjacency List vs Adjacency Matrix) : Penggunaan *Adjacency List* berbasis *Linked List* manual dipilih karena sangat hemat memori ($O(V + E)$) untuk merepresentasikan peta wilayah bencana yang bersifat renggang (*sparse graph*). *Trade-off*-nya adalah pencarian hubungan ketetanggaan jalan tidak bisa dilakukan secara instan ($O(1)$) layaknya Adjacency Matrix, melainkan harus ditelusuri secara linear sesuai derajat simpulnya.
3. Algoritma Dijkstra Tanpa Min-Heap : Pencarian lokasi jarak minimum menggunakan *nested loop* linear menghasilkan kompleksitas $O(V^2 + E)$. Desain ini dipilih karena kemudahan implementasi tanpa ketergantungan pada modul eksternal. Skenario ini sangat memadai untuk 38 titik lokasi, meskipun performanya akan menurun jika dihadapkan pada skala peta riil yang memiliki puluhan ribu simpul jalan.

Saran :

Guna meningkatkan skalabilitas, ketahanan, dan kapabilitas sistem ini di masa mendatang, beberapa poin pengembangan yang disarankan adalah:

- **Migrasi ke Binary Heap:** Mengubah struktur data *Priority Queue* berbasis *Linked List* menjadi *Min-Heap* (Binary Heap) berbasis Array, sehingga operasi penambahan dan pengambilan bantuan dapat berjalan seimbang dalam waktu logaritmik ($O(\log n)$) saat menangani ribuan antrean paket.
- **Optimasi Dijkstra dengan Fibonacci Heap:** Memodifikasi algoritma penentu rute menggunakan pencarian berbasis *Heap* untuk memangkas kompleksitas dari $O(V^2 + E)$ menjadi $O(E + V \log V)$, agar sistem siap diimplementasikan pada visualisasi peta geografis wilayah (GIS) yang sesungguhnya.
- **Penerapan Rute Dinamis (Dynamic Graph):** Mengembangkan fungsi pembaruan jalur secara *incremental* (seperti algoritma Ramalingam-Reps) agar sistem dapat merespons skenario jalan putus atau jembatan rusak secara real-time tanpa harus melakukan kalkulasi ulang graf dari awal.
- **Struktur Stack Ganda untuk Audit:** Mengintegrasikan modifikasi *Doubly Linked List* atau *Dual-Stack* pada modul Log Pengiriman agar proses audit kronologis (FIFO) pasca-bencana dapat diakses langsung dalam efisiensi ruang memori konstan ($O(1)$ space).

LAMPIRAN

Nama Branch	Pemegang Tanggung Jawab	Alokasi Fitur / Modul	Kriteria & Waktu Merge ke dev / main
main	Semua Anggota (<i>Read-Only</i>)	<i>Production-Ready Code</i> (Produk Akhir)	Hanya dapat dilakukan via <i>Pull Request</i> (PR) setelah disetujui (<i>reviewed</i>) oleh anggota lain.
dev	Semua Anggota (<i>Integrasi</i>)	Gudang penyatuan kode sebelum rilis	Dilakukan setiap akhir minggu setelah seluruh modul dipastikan lolos uji tanpa konflik.
feat/linked-list	Noomira Zakiah 25051030072	Modul 1 (Model) & Modul 2 (<i>Priority Queue</i>)	Di-merge ke dev setelah seluruh <i>unit test</i> antrean berprioritas lulus 100%.
feat/bst	M. Farras Asshidiq 25051030064	Modul 3 (<i>BST Registry</i> Lokasi)	Di-merge ke dev setelah seluruh <i>unit test</i> kamus data pohon biner lulus 100%.
feat/graph	Fahri Khairu Imtihan 25051030052	Modul 1 (<i>Graph</i>) & Modul 5 (BFS/Dijkstra)	Di-merge ke dev setelah pengujian rute terpendek dan konektivitas wilayah dinyatakan valid.
feat/cli	Restu Pamungkas 25051030080	Modul 4 (<i>Stack</i>) & Modul 6 (Simulator CLI)	Di-merge ke dev setelah integrasi menu perintah berjalan lancar secara <i>end-to-end</i> .
feat/laporan	Semua Anggota (<i>Kolaboratif</i>)	Penyusunan dokumen laporan akhir	Di-merge ke main secara bertahap sebelum tenggat waktu (<i>deadline</i>) pengumpulan Tugas Besar.

Sesuai dengan prinsip transparansi akademik, kelompok kami memanfaatkan AI (Gemini) sebagai mitra kolaborasi (*pair programmer*) dalam menyelesaikan Tugas Besar ini. Berikut adalah dokumentasi log interaksi dan pemecahan masalah bersama AI:

Log 1: Sinkronisasi Arsitektur Impor File Unit Testing

- **Konteks Masalah** : Terjadi error `ModuleNotFoundError` saat file test di folder `tests/` mencoba memanggil modul di folder `src/` karena kendala pembacaan *environment path* (terutama pada sinkronisasi folder cloud OneDrive).
- **Solusi AI** : AI merekomendasikan penggunaan pustaka `pathlib` dan `sys.path.insert(0, FOLDER_UTAMA)` secara absolut di setiap hulu file test.
- **Hasil** : Skrip `unittest` dapat dijalankan secara massal dari terminal utama tanpa kendala *path*.

Log 2: Penentuan Skenario Terburuk pada Priority Queue

- **Konteks Masalah** : Kelompok sempat keliru mengidentifikasi skenario terburuk operasi enqueue pada *Sorted Linked List*, di mana kami mengira status KRITIS selalu memicu operasi terlama.
- **Solusi AI (Koreksi)** : AI meluruskan secara matematis bahwa skenario terburuk justru terjadi jika data non-KRITIS masuk terlebih dahulu, karena memaksa pointer melakukan iterasi linear sampai ke ujung ekor Linked List. Sementara data KRITIS yang datang terakhir justru langsung masuk ke posisi head ($O(1)$).
- **Hasil** : Analisis bab Big-O di laporan menjadi valid dan akurat secara teoretis.

Log 3: Pemetaan Desain Algoritma Multi-source BFS

- **Konteks Masalah** : Kebingungan dalam menentukan pembagian wilayah depot logistik yang paling efisien tanpa harus menjalankan Dijkstra berkali-kali secara terpisah.
- **Solusi AI** : AI mendesain algoritma *Multi-source BFS* dengan memasukkan seluruh depot sekaligus ke dalam antrean *Queue* di awal iterasi, memotong kompleksitas dari $O(k \times (V^2 + E))$ menjadi hanya $O(V + E)$.
- **Hasil** : Konsep ini berhasil diadaptasi sebagai rekomendasi pengembangan sistem logistik skala besar pada bab kesimpulan.